ОТЧЁТ О ВЫПОЛНЕНИИ РАБОТЫ

PostgreSQL и Entity Framework Core

ORM, Code First, миграции и публикация на Render. Продолжение проекта CargoFlow API из лабораторной работы WT-6.

ASP.NET Core 9 · EF Core 9 · PostgreSQL 16 · Code First · Migrations · Docker · Render

СТУДЕНТ
Юшкевич Владислав

ДИСЦИПЛИНА
Веб-технологии · лабораторная WT-7

ДАТА
2 ноября 2026 г.

ТЕМА
ORM, EF Core, Code First, миграции, PostgreSQL

ИСХОДНЫЙ ПРОЕКТ
wt-lab6-6minimalapi-yushkevich

НОВЫЙ ПРОЕКТ
wt-lab7-7postgres-efcore-yushkevich

Git-репозиторий: github.com/Balur77/wt-lab7-7postgres-efcore-yushkevich
Render: wt-lab7-7postgres-efcore-yushkevich.onrender.com (адрес активируется после создания Web Service)

Отчёт составлен по фактически выполненной работе

Подпись: _____ Юшкевич В.

Цель работы и задачи

Цель работы — освоить использование **Entity Framework Core** в ASP.NET Core Minimal API для работы с **PostgreSQL**: подключить ORM, описать модель по подходу **Code First**, создать и применить миграции, заменить хранилище в памяти на базу данных и подготовить приложение к публикации на Render.

Задачи

1. Скопировать проект WT-6 и сохранить предметную область и контракт API.
2. Подключить пакеты EF Core, Npgsql и инструменты `dotnet-ef`.
3. Создать `AppDbContext` и заменить `CargoStore` на `DbSet<Cargo>`.
4. Создать миграцию `InitialCreate` и автоматически создать структуру PostgreSQL.
5. Перенести CRUD на EF Core, добавить LINQ-запросы и валидацию.
6. Изменить модель, создать миграции `AddCreatedAt`, `SeedCargoes`, `AddSupplier`.
7. Настроить автоматическое применение миграций при запуске и публикацию через Docker/Render.

> **СВЯЗЬ С WT-6**
>
> Проект WT-6 `wt-lab6-6minimalapi-yushkevich` использовал `List<Cargo>` в статическом `CargoStore`. В WT-7 этот же домен «Склад и логистика» и те же endpoint'ы работают через `AppDbContext` и PostgreSQL. Файл `Data/CargoStore.cs` удалён.

Проект	wt-lab7-7postgres-efcore-yushkevich
Предметная область	Склад и логистика, сущность `Cargo`
Стек	.NET 9, ASP.NET Core Minimal API, EF Core 9, Npgsql, PostgreSQL 16
Хранение	PostgreSQL, подход Code First, четыре миграции
Документация	`/openapi/v1.json`, `/swagger`, `/scalar/v1`
Контейнер	Docker, многоэтапная сборка, порт из `PORT`
Публикация	Render Web Service (Docker) + Render PostgreSQL

Домен и структура проекта

CargoFlow автоматизирует учёт грузов на складе. Домен и сущность `Cargo` перенесены из WT-4, WT-5 и WT-6, поэтому API продолжает уже выбранную предметную область, а не вводит новую.

Свойство	Тип / ограничение	Назначение
`Id`	`int` , PK	Идентификатор, генерируется PostgreSQL
`Name`	`varchar(100)`	Название груза
`Category`	`varchar(60)`	Категория груза
`Quantity`	`int`	Количество единиц
`Location`	`varchar(100)`	Место хранения на складе
`Weight`	`numeric(18,3)`	Вес одной единицы, кг
`Description`	`varchar(300)`	Описание груза
`IsAvailable`	`boolean`	Признак наличия на складе
`CreatedAt`	`timestamptz`	Дата добавления в БД (UTC)
`Supplier`	`varchar(120)`	Поставщик груза
`TotalWeight`	`[NotMapped]`	Общий вес партии, вычисляется
`Status`	`[NotMapped]`	Текстовый статус, вычисляется

Структура проекта

```
wt-lab7-7postgres-efcore-yushkevich/
├── Data/
│   └── AppDbContext.cs                  # DbContext, модель, HasData
├── Models/
│   └── Cargo.cs                         # сущность предметной области
├── Migrations/
│   ├── 20261101210710_InitialCreate.cs
│   ├── 20261101210747_AddCreatedAt.cs
│   ├── 20261101210825_SeedCargoes.cs
│   ├── 20261101210925_AddSupplier.cs
│   └── AppDbContextModelSnapshot.cs
├── docs/                                # отчёт и лог тестирования
├── Program.cs                           # endpoints, EF Core, Render
├── appsettings.json                     # демонстрационная строка подключения
├── Dockerfile
├── docker-compose.yml                   # PostgreSQL + API локально
└── wt-lab7-7postgres-efcore-yushkevich.csproj
```

> **Что не хранится в БД.** Свойства `TotalWeight` и `Status` помечены `[NotMapped]`: они вычисляются в C# и возвращаются в JSON, но не создают лишних столбцов в PostgreSQL.

DbContext и подключение PostgreSQL

`DbContext` — основная точка взаимодействия приложения с моделью данных и базой. `DbSet<Cargo>` представляет набор грузов, с которым выполняются операции чтения, добавления, изменения и удаления.

Пакеты и инструменты

```
dotnet add package Npgsql.EntityFrameworkCore.PostgreSQL --version 9.0.4
dotnet add package Microsoft.EntityFrameworkCore.Design --version 9.0.11
dotnet tool install --global dotnet-ef --version 9.0.11
```

Data/AppDbContext.cs

```
public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options)
        : base(options) { }

    public DbSet<Cargo> Cargoes => Set<Cargo>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<Cargo>(entity =>
        {
            entity.ToTable("Cargoes");
            entity.HasKey(cargo => cargo.Id);
            entity.Property(cargo => cargo.Name).HasMaxLength(100).IsRequired();
            entity.Property(cargo => cargo.Weight).HasPrecision(18, 3);
            entity.HasIndex(cargo => cargo.Category);
            entity.HasIndex(cargo => cargo.IsAvailable);
            entity.HasIndex(cargo => cargo.CreatedAt);
        });
    }
}
```

Регистрация в Program.cs

```
var connectionString = builder.Configuration.GetConnectionString("DefaultConnection");
var databaseUrl = builder.Configuration["DATABASE_URL"];

if (!string.IsNullOrWhiteSpace(databaseUrl))
    connectionString = BuildConnectionString(databaseUrl);

builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseNpgsql(connectionString));
```

Локально

```
{
  "ConnectionStrings": {
    "DefaultConnection":

"Host=localhost;Port=5432;Database=wt_lab7;Username=postg
res;Password=***"
  }
}
```

Render

```
DATABASE_URL =
postgres://user:***@host:5432/database

Internal Database URL
→ SSL Mode=Disable
внешний адрес → TLS
```

> Настоящий пароль не хранится в Git. В учебном `appsettings.json` оставлена только демонстрационная строка локального контейнера; для Render значение передаётся переменной окружения `DATABASE_URL`.

Миграции: InitialCreate и AddCreatedAt

При подходе **Code First** источником модели является C#-код. EF Core сравнивает текущую модель с предыдущим состоянием и формирует миграцию — инструкцию изменения структуры базы данных.

Миграция	Что изменяет
`InitialCreate`	Создаёт таблицу `Cargoes`, первичный ключ и индексы
`AddCreatedAt`	Добавляет столбец `CreatedAt` и индекс по дате
`SeedCargoes`	Добавляет шесть начальных грузов через `HasData`
`AddSupplier`	Добавляет столбец `Supplier` и заполняет его

Создание и применение

```
dotnet ef migrations add InitialCreate
dotnet ef migrations add AddCreatedAt
dotnet ef migrations add SeedCargoes
dotnet ef migrations add AddSupplier

dotnet ef migrations list
dotnet ef database update
```

Фрагмент InitialCreate

```
migrationBuilder.CreateTable(
    name: "Cargoes",
    columns: table => new
    {
        Id = table.Column<int>(type: "integer", nullable: false)
            .Annotation("Npgsql:ValueGenerationStrategy",
                NpgsqlValueGenerationStrategy.IdentityByDefaultColumn),
        Name = table.Column<string>(type: "character varying(100)",
            maxLength: 100, nullable: false),
        Category = table.Column<string>(type: "character varying(60)",
            maxLength: 60, nullable: false),
        Quantity = table.Column<int>(type: "integer", nullable: false),
        Location = table.Column<string>(type: "character varying(100)",
            maxLength: 100, nullable: false),
        Weight = table.Column<decimal>(type: "numeric(18,3)",
            precision: 18, scale: 3, nullable: false),
        Description = table.Column<string>(type: "character varying(300)",
            maxLength: 300, nullable: false),
        IsAvailable = table.Column<bool>(type: "boolean", nullable: false)
    },
    constraints: table =>
        table.PrimaryKey("PK_Cargoes", x => x.Id));
```

Список миграций после выполнения

```
$ dotnet ef migrations list
20261101210710_InitialCreate
20261101210747_AddCreatedAt
20261101210825_SeedCargoes
20261101210925_AddSupplier
```

> **ВАЖНО**
>
> `EnsureCreated()` не используется. Структура базы создаётся только через `Database.Migrate()`, иначе EF Core обойдёт систему миграций и не сможет корректно применять последующие изменения.

Начальные данные и изменение модели

После первой миграции модель была изменена: добавлены `CreatedAt` и `Supplier`. Для каждой изменения создана отдельная миграция, а начальные данные добавлены через `HasData`, а не вручную SQL-запросами.

HasData в AppDbContext

```
entity.HasData(
    new Cargo
    {
        Id = 1,
        Name = "Ноутбуки Dell Latitude 5440",
        Category = "Электроника",
        Quantity = 24,
        Location = "Сектор А-1, стеллаж 3",
        Weight = 2.4m,
        Description = "Партия офисных ноутбуков...",
        IsAvailable = true,
        CreatedAt = new DateTime(2026, 1, 10, 9, 0, 0, DateTimeKind.Utc),
        Supplier = "Dell Technologies"
    },
    // ... ещё пять грузов
);
```

AddSupplier: изменение уже существующей таблицы

```
migrationBuilder.AddColumn<string>(
    name: "Supplier",
    table: "Cargoes",
    type: "character varying(120)",
    maxLength: 120,
    nullable: false,
    defaultValue: "");

migrationBuilder.UpdateData(
    table: "Cargoes",
    keyColumn: "Id",
    keyValue: 1,
    column: "Supplier",
    value: "Dell Technologies");
```

Автоматическое применение при запуске

```
using (var scope = app.Services.CreateScope())
{
    var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();
    db.Database.Migrate();
}
```

```
Запуск приложения
        ↓
Подключение к PostgreSQL
        ↓
Проверка __EFMigrationsHistory
        ↓
Применение отсутствующих миграций
        ↓
Запуск HTTP API
```

Изменение модели без миграции. Если добавить свойство в C# и не создать миграцию, EF Core будет ожидать столбец, которого нет в базе, и запросы завершатся ошибкой. Поэтому изменение модели всегда сопровождается `dotnet ef migrations add` и коммитом файлов миграции.

Структура PostgreSQL

После `dotnet ef database update` в базе `wt_lab7` появились таблица предметных данных `Cargoes` и служебная таблица истории миграций `__EFMigrationsHistory`.

Таблицы в схеме public

```
SELECT table_name
FROM information_schema.tables
WHERE table_schema = 'public'
ORDER BY table_name;

      table_name
----------------------
 Cargoes
 __EFMigrationsHistory
```

Структура таблицы Cargoes

```
 column_name |        data_type         | is_nullable
-------------+--------------------------+-------------
 Id          | integer                  | NO
 Name        | character varying        | NO
 Category    | character varying        | NO
 Quantity    | integer                  | NO
 Location    | character varying        | NO
 Weight      | numeric                  | NO
 Description | character varying        | NO
 IsAvailable | boolean                  | NO
 CreatedAt   | timestamp with time zone | NO
 Supplier    | character varying        | NO
```

История миграций

```
         MigrationId          | ProductVersion
------------------------------+----------------
 20261101210710_InitialCreate | 9.0.11
 20261101210747_AddCreatedAt  | 9.0.11
 20261101210825_SeedCargoes   | 9.0.11
 20261101210925_AddSupplier   | 9.0.11
```

Начальные данные

```
 Id |               Name                 |    Category     | Weight |     Supplier
----+------------------------------------+-----------------+--------+------------------
  1 | Ноутбуки Dell Latitude 5440        | Электроника     |  2.400 | Dell Technologies
  2 | Офисные кресла ErgoPro             | Мебель          | 12.500 | ErgoPro
  3 | Кофемашины Jura E8                 | Бытовая техника |  9.800 | Jura
  4 | Наборы строительного инструмента   | Инструменты     | 25.000 | ИнструментТорг
  5 | Медицинские маски                  | Медицина        |  0.005 | МедСнаб
  6 | Серверное оборудование Supermicro  | Электроника     | 18.600 | Supermicro
```

ПРОВЕРКА SQL

Запрос `SELECT "Name", "Weight" FROM "Cargoes" WHERE "Weight" > 10` соответствует LINQ-запросу `db.Cargoes.Where(c => c.Weight > 10)`. EF Core преобразует LINQ в SQL и выполняет фильтрацию на стороне PostgreSQL.

Операции с данными

Все endpoint'ы получают `AppDbContext` через параметры Minimal API. `CargoStore` удалён: чтение и запись выполняются через `DbSet<Cargo>` и `SaveChangesAsync()`.

Чтение списка и объекта

```
app.MapGet("/api/cargoes", async (AppDbContext db) =>
{
    var cargoes = await db.Cargoes.AsNoTracking().ToListAsync();
    return Results.Ok(cargoes);
});

app.MapGet("/api/cargoes/{id:int}", async (int id, AppDbContext db) =>
{
    var cargo = await db.Cargoes.AsNoTracking()
        .FirstOrDefaultAsync(x => x.Id == id);
    return cargo is null ? Results.NotFound() : Results.Ok(cargo);
});
```

Создание

```
app.MapPost("/api/cargoes", async (Cargo input, AppDbContext db) =>
{
    Normalize(input);
    var errors = Validate(input);
    if (errors.Count > 0) return Results.ValidationProblem(errors);

    input.Id = 0;
    input.CreatedAt = DateTime.UtcNow;
    db.Cargoes.Add(input);
    await db.SaveChangesAsync();

    return Results.Created($"/api/cargoes/{input.Id}", input);
});
```

Изменение и удаление

```
app.MapPut("/api/cargoes/{id:int}", async (int id, Cargo input, AppDbContext db) =>
{
    var cargo = await db.Cargoes.FirstOrDefaultAsync(x => x.Id == id);
    if (cargo is null) return Results.NotFound();

    cargo.Name = input.Name;
    cargo.Category = input.Category;
    cargo.Quantity = input.Quantity;
    cargo.Location = input.Location;
    cargo.Weight = input.Weight;
    cargo.Description = input.Description;
    cargo.IsAvailable = input.IsAvailable;
    cargo.Supplier = input.Supplier;

    await db.SaveChangesAsync();
    return Results.Ok(cargo);
});

app.MapDelete("/api/cargoes/{id:int}", async (int id, AppDbContext db) =>
{
    var cargo = await db.Cargoes.FirstOrDefaultAsync(x => x.Id == id);
    if (cargo is null) return Results.NotFound();

    db.Cargoes.Remove(cargo);
    await db.SaveChangesAsync();
    return Results.NoContent();
});
```

> EF Core отслеживает изменения загруженных сущностей. Для `PUT` достаточно изменить свойства и вызвать `SaveChangesAsync()` — ORM сам сформирует `UPDATE`. Для `DELETE` — `Remove()` и `SaveChangesAsync()`.

Запросы LINQ и SQL

В проекте реализовано больше трёх LINQ-запросов: фильтрация, сортировка, выборка с условием, группировка с агрегатами и выборка по дате. EF Core транслирует их в SQL и выполняет в PostgreSQL.

Фильтрация

```
var heavy = await db.Cargoes
    .Where(cargo => cargo.Weight > 10)
    .ToListAsync();
```

Сортировка

```
var byWeight = await db.Cargoes
    .OrderBy(cargo => cargo.Weight)
    .ToListAsync();
```

Выборка с условием

```
var available = await db.Cargoes
    .Where(cargo => cargo.IsAvailable)
    .OrderBy(cargo => cargo.Name)
    .ToListAsync();
```

Собственный запрос: группировка и агрегаты

```
var stats = await db.Cargoes
    .GroupBy(cargo => cargo.Category)
    .Select(group => new
    {
        Category = group.Key,
        Count = group.Count(),
        TotalQuantity = group.Sum(cargo => cargo.Quantity),
        TotalWeight = group.Sum(cargo => cargo.Weight * cargo.Quantity),
        AvailableCount = group.Count(cargo => cargo.IsAvailable)
    })
    .OrderByDescending(item => item.TotalWeight)
    .ToListAsync();
```

LINQ и соответствующий SQL

LINQ	SQL
`Where(c => c.Weight > 100)`	`WHERE "Weight" > 100`
`OrderBy(c => c.Weight)`	`ORDER BY "Weight"`
`Where(c => c.IsAvailable)`	`WHERE "IsAvailable" = TRUE`
`GroupBy(c => c.Category)`	`GROUP BY "Category"`
`Count()`, `Sum()`, `Average()`	`COUNT()`, `SUM()`, `AVG()`

> **ОТЛИЧИЕ ОТ WT-6**
>
> В WT-6 фильтрация выполнялась над `List<Cargo>` в памяти. В WT-7 `db.Cargoes.Where(...)` формирует SQL-запрос, который выполняется PostgreSQL. Это уменьшает объём передаваемых данных и использует индексы базы.

LINQ-endpoint'ы: `/api/cargoes/available`, `/api/cargoes/heavy?minWeight=10`, `/api/cargoes/stats`, `/api/cargoes/recent?days=30`.

Docker и Render

Приложение собирается в Docker-образ по тому же принципу, что и в WT-6, но теперь при старте подключается к PostgreSQL и автоматически применяет миграции. На Render строка подключения передаётся через `DATABASE_URL`.

Dockerfile

```
FROM mcr.microsoft.com/dotnet/sdk:9.0 AS build
WORKDIR /src
COPY wt-lab7-7postgres-efcore-yushkevich.csproj ./
RUN dotnet restore
COPY . .
RUN dotnet publish wt-lab7-7postgres-efcore-yushkevich.csproj \
    -c Release -o /app/publish /p:UseAppHost=false

FROM mcr.microsoft.com/dotnet/aspnet:9.0 AS final
WORKDIR /app
COPY --from=build /app/publish .
ENV ASPNETCORE_URLS=http://0.0.0.0:10000
ENV PORT=10000
EXPOSE 10000
ENTRYPOINT ["dotnet", "wt-lab7-7postgres-efcore-yushkevich.dll"]
```

Локальная проверка Docker Compose

```
docker compose up --build
# API:        http://localhost:10000
# PostgreSQL: localhost:5432, база wt_lab7
```

Публикация на Render

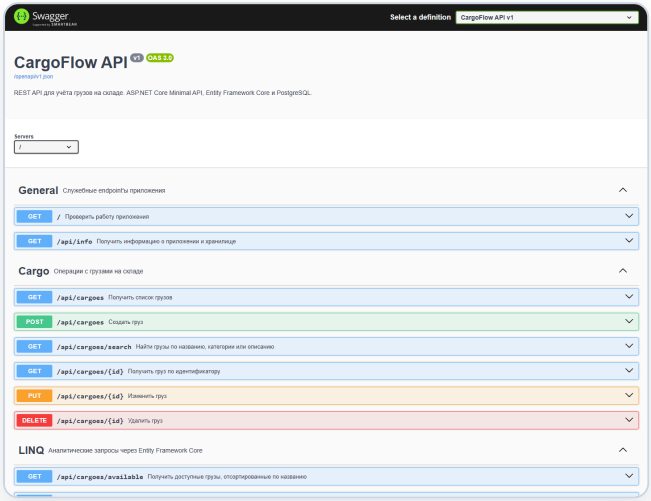
1. Создать PostgreSQL: **New → PostgreSQL**, имя `wt-lab7-db-yushkevich`.
2. Создать **Web Service** из GitHub-репозитория WT-7, Runtime — **Docker**.
3. В **Environment** добавить `DATABASE_URL` = **Internal Database URL**.
4. Выполнить Deploy и дождаться статуса **Live**.
5. Проверить `GET /api/info` и `GET /api/cargoes`.

```
Application starting
        ↓
Connection to PostgreSQL
        ↓
EF Core
        ↓
Migrations
        ↓
Application started
```

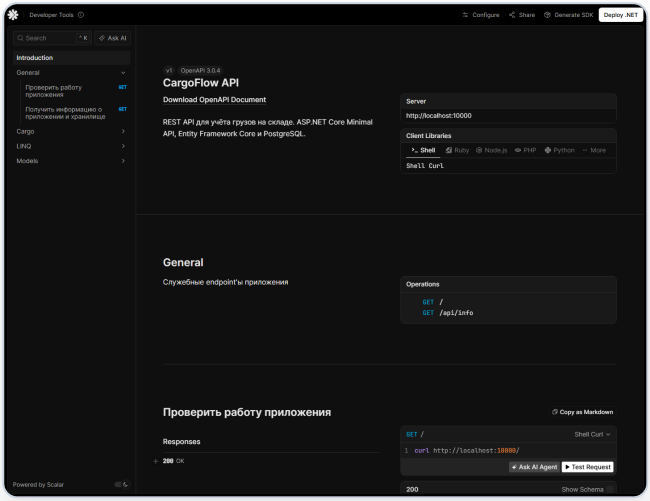
> **Секреты.** Пароль PostgreSQL не хранится в `Program.cs`, `appsettings.json` или Git-репозитории. На Render значение `DATABASE_URL` задаётся только в настройках Web Service. В отчёте и README полные строки подключения не публикуются.

Проверка REST API

API проверено локально после применения миграций: данные читаются из PostgreSQL, полный цикл CRUD изменяет записи в базе, а документация OpenAPI/Swagger/Scalar доступна по относительному адресу.

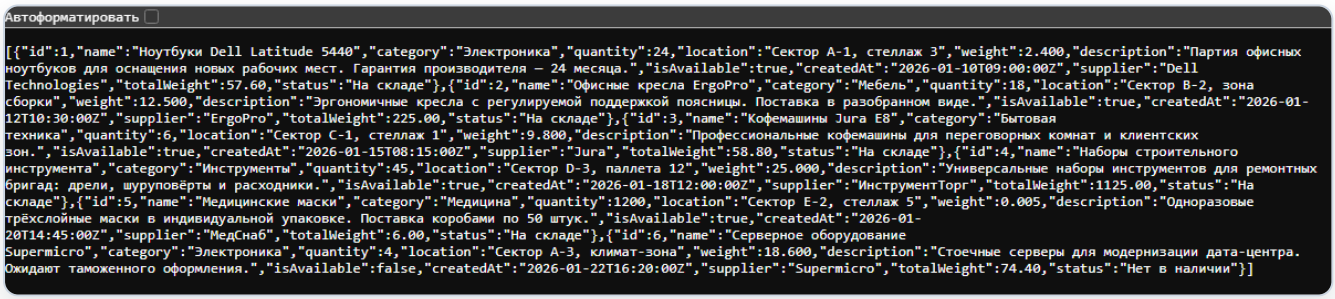


Swagger UI: все endpoint'ы и схемы



Scalar UI: интерактивная документация

Ответ GET /api/cargoes из PostgreSQL



JSON-ответ: шесть грузов, поля `createdAt` и `supplier`

Запрос	Результат	Статус
`GET /api/cargoes`	6 записей из PostgreSQL	**200 OK**
`GET /api/cargoes/1`	объект найден	**200 OK**
`GET /api/cargoes/999`	объект отсутствует	**404 Not Found**
`GET /api/cargoes/search?search=ноут`	1 запись	**200 OK**
`POST /api/cargoes`	создан `id = 7`	**201 Created**
`POST` с пустыми полями	ошибки валидации	**400 Bad Request**
`PUT /api/cargoes/7`	данные обновлены	**200 OK**
`DELETE /api/cargoes/7`	запись удалена	**204 No Content**
`GET /api/cargoes/7` после удаления	объект отсутствует	**404 Not Found**

Итоги работы

Проект WT-6 переведён с хранения в памяти на PostgreSQL через Entity Framework Core. Предметная область, модель `Cargo` и контракт REST API сохранены, а слой данных полностью заменён на ORM.

Что выполнено

- ✓ Подключены EF Core, Npgsql и `dotnet-ef`.
- ✓ Создан `AppDbContext` с `DbSet<Cargo>`.
- ✓ Удалён `Data/CargoStore.cs`; CRUD переведён на EF Core.
- ✓ Созданы и применены четыре миграции Code First.
- ✓ Структура PostgreSQL создана автоматически, история — в `__EFMigrationsHistory`.
- ✓ Добавлены начальные данные через `HasData`.
- ✓ Реализованы LINQ-запросы: фильтрация, сортировка, группировка, выборка по дате.
- ✓ Настроено автоматическое применение миграций при запуске.
- ✓ Подготовлены Dockerfile, Docker Compose и конфигурация Render через `DATABASE_URL`.
- ✓ Проверены все CRUD-операции, HTTP-статусы и документация API.

Преимущества ORM

Code First
Модель описывается в C#, а не в SQL-скриптах

Миграции
История изменений схемы хранится в Git

LINQ
Запросы проверяются компилятором

PostgreSQL
Данные сохраняются между перезапусками

> **РЕЗУЛЬТАТ**
> Приложение собирается, запускается, подключается к PostgreSQL, применяет миграции и возвращает данные из базы. Для публикации на Render достаточно указать `DATABASE_URL` и выполнить Deploy; при каждом новом `git push` Render пересобирает контейнер и применяет новые миграции.

Ссылки

GitHub WT-6	github.com/Balur77/wt-lab6-6minimalapi-yushkevich
GitHub WT-7	github.com/Balur77/wt-lab7-7postgres-efcore-yushkevich
Render	wt-lab7-7postgres-efcore-yushkevich.onrender.com
Лог тестирования	`docs/api-testing.md`

Ключевые вопросы

Вопрос	Ответ
Что такое ORM?	Слой, который отображает объекты C# на таблицы реляционной БД и берёт на себя формирование SQL.
Что такое `DbContext`?	Сессия работы с БД: набор сущностей, отслеживание изменений, `SaveChanges`, транзакции.
Что такое `DbSet`?	Набор сущностей одного типа, через который выполняются запросы и операции добавления/удаления.
Что означает Code First?	Модель и схема БД выводятся из C#-классов; миграции создаются по изменениям модели.
Что такое миграция?	Файл с инструкциями изменения схемы БД между двумя состояниями модели.
Зачем `migrations add`?	Сравнить модель с предыдущим снимком и сгенерировать код изменения схемы.
Зачем `database update`?	Применить неприменённые миграции к базе и записать их в историю.
Где хранится история?	В таблице `__EFMigrationsHistory`.
`EnsureCreated` и `Migrate`?	`EnsureCreated` создаёт схему без истории; `Migrate` применяет миграции. Вместе их использовать нельзя.
Что делает `Database.Migrate()`?	Применяет все отсутствующие миграции при старте приложения.
Что такое `DATABASE_URL`?	Переменная окружения Render со строкой подключения к PostgreSQL.
Почему пароль не в Git?	Репозиторий может быть публичным; секреты передаются через переменные окружения.
Что будет, если модель изменена без миграции?	Схема БД не совпадёт с моделью, и EF Core завершит запросы ошибкой отсутствующего столбца.

ВЫВОД

Entity Framework Core позволил сохранить структуру Minimal API и предметную модель WT-6, заменив только слой хранения. Миграции обеспечивают воспроизводимое создание и обновление PostgreSQL как локально, так и на Render.